FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería en Tecnologías de la Información Asignatura: Asignatura: Metodología de Desarrollo de Software NRC: 30693
	Apellidos y nombres del estudiante: JARAMILLO APUNTE CHRISTOPHER MATEO

ANTECEDENTES

Históricamente, el desarrollo de software carecía de procesos formales, lo que generaba productos de baja calidad. A partir de las propuestas de Pressman y la creación de estándares internacionales, la ingeniería de software se consolidó como una disciplina basada en metodologías estructuradas y normas de calidad. Como hemos visualizado con anterioridad desde la mención en 1966 hasta la actualidad se han desarrollado nuevas metodologías y técnicas para mejorar los procesos correspondientes.

OBJETIVO

Objetivo general: Analizar los fundamentos de la Ingeniería de Software y sus metodologías mediante la revisión de fuentes bibliográficas para fundamentar la toma de decisiones en el desarrollo de sistemas de información.

Objetivos específicos:

- Identificar las fases del ciclo de vida de software y las actividades estructurales que garantizan un proceso organizado.
- Diferenciar entre metodologías ágiles (como XP) y modelos prescriptivos basándonos en sus enfoques filosóficos y operativos.
- Examinar los estándares internacionales de calidad (ISO/IEC 25010) para evaluar las características técnicas de un producto de software.

DESARROLLO

1. **¿Qué es la metodología de desarrollo de software?**
   Un conjunto de métodos que cubren todo el ciclo de vida de desarrollo de sistemas y que están unidos por un enfoque general filosófico.

   Justificación: Según Pressman, la metodología no se limita a pasos técnicos, sino que es un marco de trabajo integral que incluye tareas, hitos y productos bajo una filosofía de gestión específica. Esto justifica que sea un "conjunto de métodos que cubren todo el ciclo de vida" bajo un enfoque filosófico. Esto lo menciona en capítulo 1 y Capítulo 2.
   Referencia en el texto: (Pressman, 2010, p. 12 y 27).

2. **Las actividades estructurales del proceso de desarrollo de Software**
   Despliegue - Comunicación - Modelado - Diseño

   Justificación: Según Pressman, un proceso de software incluye un conjunto de actividades estructurales aplicables a todos los proyectos. La Comunicación es esencial para entender los objetivos de los interesados; el Modelado permite crear representaciones (como el diseño) para entender el panorama general; y el Despliegue entrega el software al cliente para su evaluación. Estas actividades forman un ciclo lógico que garantiza que el desarrollo sea organizado y no caótico.
   Referencia en el texto: (Pressman, 2010, pp. 12-13).

3. **Una de las razones por la cual se incrementan los costos al agregar funcionalidad después que un sistema está en operación es:**
Práctica de desarrollo deficiente

Justificación: Según Sommerville, una práctica de desarrollo deficiente ocurre cuando el equipo de desarrollo no tiene incentivos para escribir software mantenible (por ejemplo, cuando el contrato de mantenimiento se le dará a otra empresa). Esto lleva a tomar "atajos" técnicos que ahorran tiempo inicial pero aumentan los costos a largo plazo.
Referencia en el texto: (Sommerville, I., 2011, pp. 245).

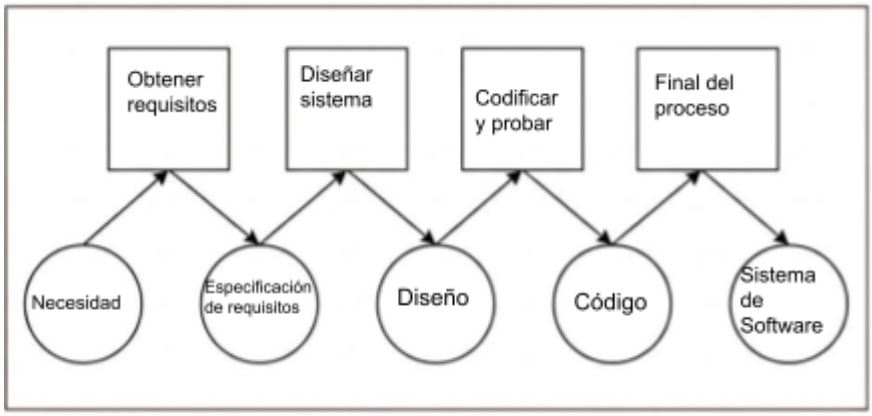
4. **Ingeniería de software asistida por computador significa:**
Conjunto de herramientas de hardware integradas para ser utilizadas por otras

Justificación: Se define como el uso de herramientas computacionales (software y su soporte) para asistir en las actividades de ingeniería de software, como el análisis de requerimientos, el diseño y la generación de código. Según Pressman (2010), estas herramientas integran el proceso, los métodos y el soporte automatizado para crear un entorno de desarrollo eficiente. Las herramientas CASE son un conjunto de programas (software) y métodos integrados que automatizan las actividades del ciclo de vida del desarrollo.
Referencia en el texto: (Pressman, 2010, p. 13).

5. **La metodología con sus tipos correspondientes de clasificación es:**
Metodología XP (orientada a objetos-ágil)

Justificación: XP es una metodología ágil porque se basa en ciclos cortos de desarrollo (iteraciones) y respuesta rápida al cambio. Es "orientada a objetos" porque utiliza este paradigma de programación para facilitar la modularidad y reutilización del código. Pressman clasifica a XP como el modelo ágil más utilizado que enfatiza la participación del cliente y la prueba continua.
Referencia en el texto: (Pressman, 2010, Capítulo 3).

6. **En gráfico propuesto, la sentencia ordenada de las figuras es:**



Necesidad, obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software.

Justificación: Esta secuencia representa el Ciclo de Vida de Desarrollo de Sistemas (SDLC) tradicional. El orden lógico (Necesidad - Requisitos - Diseño - Codificación - Pruebas) asegura que no se construya nada sin una especificación previa, reduciendo errores y costos. Sommerville explica el "Modelo en Cascada" o lineal, donde cada fase debe completarse antes de pasar a la siguiente.
Referencia en el texto: (Sommerville, 2011, Capítulo 2).

7. **Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo de Software son:**
ISO/TEC 25010 producto software

Justificación: Divide la calidad en dos partes: Calidad del Producto (características internas y externas) y Calidad en el Uso. Pressman trata el tema de la calidad no solo menciona la norma ISO, sino que explica la filosofía detrás de ella. Dice que la calidad es un "proceso eficaz que aplica métodos y herramientas, produce productos de trabajo de alta calidad y se mide mediante estándares".

Referencia en el texto: (Pressman, 2010, Capítulo 14)

8. **La norma ISO/IEC 25010 identifica características y subcaracterísticas de la calidad del producto software. La opción correcta es:**
   La eficiencia del desempleo es característica.

   Justificación: La Eficiencia de desempeño es una de las 8 características principales de la ISO 25010. Se refiere a la capacidad del software para comportarse según los tiempos de respuesta y el uso de recursos bajo condiciones determinadas.
   Sommerville es más detallado con las normas ISO, diferencia claramente entre la calidad del proceso (cómo lo fábricas) y la calidad del producto (cómo funciona). Justifica el uso de ISO como una forma de dar confianza al cliente de que el software es fiable.
   Referencia en el texto: (Sommerville, 2011, Capítulo 24) ; ISO/IEC 25010, Sección de Modelo de Calidad del Producto.

9. **Según los roles y las responsabilidades correspondientes, qué opción es correcta.**
   Verificar los cambios y resolver conflictos corresponde a un desarrollador.

   Justificación:
   Dentro del flujo de trabajo de la gestión de configuración y el desarrollo colaborativo, el desarrollador es el responsable directo de integrar su código en el espacio de trabajo común. Este proceso implica que, al realizar un "merge" o integración, el desarrollador debe verificar que sus cambios no rompan la funcionalidad existente y es su responsabilidad técnica resolver cualquier conflicto que surja cuando dos personas modifican el mismo componente. Según la ingeniería de software moderna, mientras que el Configuration Manager establece las reglas, es el desarrollador quien ejecuta la resolución técnica de conflictos en el día a día (Pressman, 2010).

10. **Según los objetivos con la correspondiente fase del proceso unificado de desarrollo, qué opción es la correcta.**
    Inicio establece el alcance del proyecto y las condiciones de los límites, criterios de aceptación, lo que se necesita en el producto y lo que no.

    Justificación
    La fase de Inicio (Inception) en el Proceso Unificado (RUP) tiene como objetivo principal determinar la viabilidad del proyecto. En esta etapa no se desarrolla el software, sino que se define el modelo de negocio y el alcance del sistema. Es aquí donde se identifican los límites (qué hará y qué no hará el sistema) y se acuerdan los criterios de éxito con los interesados para asegurar que todos tengan la misma visión antes de invertir más recursos en las fases de Elaboración y Construcción (Sommerville, 2011).

CONCLUSIONES

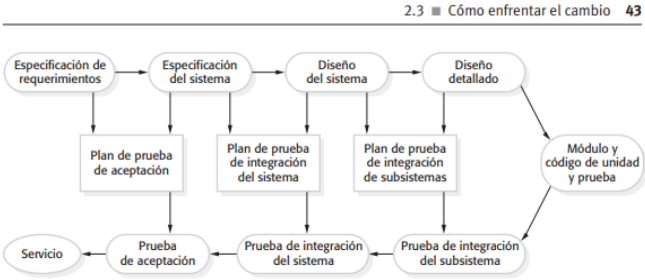
1. Se identifican diferente tipos de metodología funcionales y eficientes, mismas que se aplican según el proyecto y necesidades específicas requeridas.
2. La Ingeniería de Software se consolida como una disciplina formal necesaria para mitigar los riesgos de la "crisis del software", permitiendo que los sistemas sean escalables, mantenibles y de alta calidad mediante procesos estandarizados.
3. El modelo de desarrollo incremental destaca como una alternativa superior a los modelos lineales en entornos dinámicos, ya que permite la entrega temprana de valor al cliente y una mayor flexibilidad ante el cambio de requerimientos.

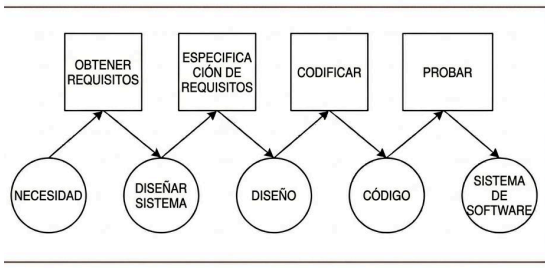
RECOMENDACIONES

1. Seleccionar el modelo de proceso Cascada, Incremental o Evolutivo, tras un análisis exhaustivo de la estabilidad de los requisitos, priorizando métodos ágiles o incrementales cuando la incertidumbre sea elevada.
2. Fortalecer la fase de Especificación de Requisitos mediante una comunicación constante con los interesados, dado que la mayoría de los fallos en el software derivan de una definición ambigua de las necesidades del usuario.
3. Fomentar la actualización continua en estándares internacionales de calidad (como la norma ISO/IEC 25010) para asegurar que el software desarrollado no sólo funcione, sino que cumpla con atributos de seguridad y eficiencia.

Sangolquí, a 13 de abril de 2026

ANEXOS (opcionales/RUBRICA DE LA GUIA)





INTEGRANTES	
N°	**Apellidos y Nombres**
1	Christopher Mateo Jaramillo Apunte
2	Soza Morales Melany Valeria
3	Velasquez Alay Mery Stephanie
4	Verdezoto Armijos Miguel Alejandro

BIBLIOGRAFÍAS

1. Libro de Roger Pressman (El de las actividades estructurales y metodología):

Pressman, R. S., & Maxim, B. R. (2010). *Ingeniería del software: Un enfoque práctico* (7ma ed.). McGraw-Hill Interamericana.

2. Libro de Ian Sommerville (El de las prácticas deficientes y procesos):

Sommerville, I. (2011). *Ingeniería de software* (9na ed.). Pearson Educación.

3. Norma Internacional (La de calidad del producto):

Organización Internacional de Normalización [ISO]. (2011). *Sistemas e ingeniería de software — Requisitos y evaluación de calidad de sistemas y software (SQuaRE) — Modelos de calidad de sistema y software* (ISO/IEC 25010:2011).